

Machine Learning Esonero

Mattia Cosimi

December 2024

1 Fondamenti di ML

1.1 Dati e Pattern

I dati sono un ingrediente fondamentale del machine learning dove il comportamento degli algoritmi non è pre-programmato ma appreso dai dati stessi. Utilizzeremo spesso il termine **Pattern** per riferirci ai dati. La disciplina che studia il riconoscimento dei pattern prende il nome di **Pattern Recognition**.

1.1.1 Tipi di Pattern

Numerici: valori associati a caratteristiche misurabili o conteggi.

Sono rappresentabili naturalmente come vettori numerici nello spazio multidimensionale. L'estrazione di caratteristiche da segnali produce vettori numerici detti anche feature vectors. Es. Persona [altezza, circonferenza toracica, circonferenza fianchi, lunghezza del piede]

Categorici: valori associati a caratteristiche qualitative e alla presenza/assenza di una caratteristica (yes/no value).

Non sono mappabili in valori numerici. Es. Persona: [sesso, maggiorenne, colore occhi, gruppo sanguigno]. Solitamente vengono gestiti da sistemi a regole e alberi di classificazione. Molto utilizzati nell'ambito del **data mining**.

Sequenze: pattern sequenziali con relazioni spaziali o temporali.

Ne sono un esempio, uno stream audio (sequenza di suoni) corrispondente alla pronuncia di una parola, una frase (sequenza di parole) in linguaggio naturale, l'andamento di un titolo di borsa (sequenza temporale del prezzo di chiusura). La posizione nella sequenza e le relazioni con predecessori e successori sono importanti

Altri dati strutturati: output organizzati in strutture complesse quali alberi e grafi. Esempio nella traduzione di una frase in linguaggio naturale, l'output desiderato è l'insieme dei parse tree plausibili.

1.1.2 Classificazione

Assegna una **classe** a un pattern. Risulta necessario apprendere una funzione capace di eseguire il mapping dallo spazio dei pattern allo spazio delle classi. Si usa spesso anche il termine riconoscimento. Nel caso di 2 sole classi si usa il termine **binary classification**, con più di due classi **multi-class classification**.

Esempi nel problema di classificazione: spam detection, face recognition...

Una classe è un insieme di pattern aventi proprietà comuni. Il concetto di classe è semantico e dipende strettamente dall'applicazione, es. 21 classi per il riconoscimento di lettere dell'alfabeto, 2 classi per distinguere le lettere dell'alfabeto italiano da quello cirillico.

1.1.3 Regressione

Assegna un **valore continuo** a un pattern. Risulta utile per la predizione di valori continui. Risolvere un problema di regressione corrisponde ad apprendere una funzione approssimante delle coppie $\langle \text{input}, \text{output} \rangle$ date.

Es. stima dell'altezza di una persona in base al peso o stima prezzi vendita appartamenti nel mercato immobiliare.

1.1.4 Clustering

Individua **gruppi** (cluster) di pattern con caratteristiche simili. Le classi del problema non sono note (apprendimento non supervisionato) e i pattern non etichettati, la natura non supervisionata del problema lo rende più complesso della classificazione. I cluster individuati nell'apprendimento possono essere poi utilizzati come classi.

Es. Marketing: definizione di gruppi utenti in base ai consumi, Genetica: raggruppamento individui sulla base analogie DNA.

1.1.5 Riduzione dimensionalità

Riduce il numero di dimensioni dei pattern in input. Consiste nell'apprendimento di un mapping da R^d a R^k (con $k < d$). L'operazione comporta una perdita di informazione. L'obiettivo è conservare le informazioni importanti.

1.1.6 Representation Learning

Il successo di molte applicazioni di machine learning dipende dall'efficacia di rappresentazione dei pattern in termini di features. La definizione di features ad-hoc per le diverse applicazioni prende il nome di **feature engineering**. Ad esempio per il riconoscimento di oggetti esistono numerosi descrittori di forma, colore e tessitura che possiamo utilizzare per convertire immagini in vettori numerici. Volendo si potrebbe apprendere anche da raw data e un esempio sono proprio le tecniche di **Deep Learning** (es. convolutional neural networks) che utilizzano in input raw data ed estraggono automaticamente da essi le feature necessarie per risolvere il problema d'interesse.

1.1.7 Problemi di Computer Vision

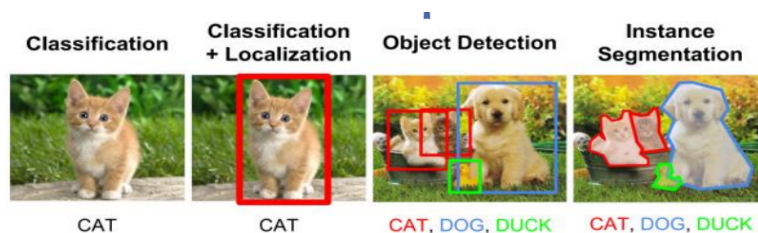


Figure 1

Classification: determina la classe dell'oggetto. **Classification e Localization:** determina la classe dell'oggetto e la sua posizione nell'immagine. **Detection:** per ogni oggetto determina la classe e la posizione nell'immagine. **Segmentation:** al posto di una bounding box etichetta i singoli pixel dell'immagine a classi con indici delle classi.

1.2 Apprendimento

1.2.1 Supervisionato (supervised)

Sono note le classi dei pattern utilizzati per l'addestramento. Il training set è etichettato. Situazione tipica nella classificazione, regressione e in alcune tecniche di riduzione di dimensionalità.

1.2.2 Non supervisionato (Unsupervised)

Non sono note le classi dei pattern utilizzati per l'addestramento. Il training set non è etichettato. Situazione tipica nel clustering e nella maggior parte di tecniche di riduzione di dimensionalità.

1.2.3 Semi-Supervisionato (Semi-Supervised)

Il training set è etichettato parzialmente, la distribuzione dei pattern non etichettati può aiutare a ottimizzare la regola di classificazione.

1.2.4 Batch

L'addestramento è effettuato una sola volta su un training set dato. Una volta terminato il training, il sistema passa in <<Working mode>> e non è in grado di apprendere ulteriormente.

1.2.5 Incrementale

A seguito dell'addestramento iniziale, sono possibili ulteriori sessioni di addestramento, es. sequenze di Batch, il rischio principale è il catastrofic forgetting ovvero il sistema dimentica quello che ha appreso in precedenza.

1.2.6 Naturale

Addestramento continuo (per tutta la vita), coesistenza di approccio supervisionato e non supervisionato

1.2.7 Reinforcement Learning

L'obiettivo è apprendere un comportamento ottimale a partire dalle esperienze passate. Un agente esegue **azioni** che modificano l'**ambiente**, provocando passaggi da uno **stato** all'altro. Quando l'agente ottiene risultati positivi riceve una ricompensa (**reward**) che però può essere temporalmente ritardata rispetto all'azione, o alla sequenza di azioni, che l'hanno determinata. Obiettivo è apprendere l'azione ottimale in ciascun stato, in modo da massimizzare la somma dei reward ottenuti nel lungo periodo.

Trova numerose applicazioni in **Robotica**, una sua estensione (deep) denominata **Deep Reinforcement Learning (DRL)** è alla base dei successi ottenuti da Google DeepMind

2 Parametri e Funzione Obiettivo

2.1 Funzione obiettivo

In genere, il comportamento di un algoritmo di Machine Learning è regolato da un set di parametri θ . L'apprendimento consiste nel determinare il valore ottimo θ^* di questi parametri. Dato un training set **Train** e un insieme di parametri, la **funzione obiettivo** $f(\text{Train}, \theta)$ può indicare:

L'**ottimalità** della soluzione (da massimizzare):

$$\theta^* = \operatorname{argmax}_{\theta} f(\text{Train}, \theta)$$

Oppure l'errore o perdita (**loss-function**) da minimizzare.

$$\theta^* = \operatorname{argmin}_{\theta} f(\text{Train}, \theta)$$

$f(\text{Train}, \theta)$ può essere ottimizzata: **esplicitamente**, con metodi che operano a partire dalla sua definizione matematica. Es. si calcolano le derivate parziali di f rispetto ai parametri (gradiente), si eguaglia il gradiente a 0 (sistema di equazioni) e si risolve rispetto ai parametri. O **implicitamente**, utilizzando euristiche che modificano i parametri in modo coerente con f . Es. clustering con algoritmo k-means.

2.2 Iperparametri

Molti algoritmi richiedono di definire, prima dell'apprendimento vero e proprio, il valore dei cosiddetti iperparametri H .

Sono un esempio: il numero di neuroni di una rete neurale, il numero di vicini k in un classificatore k -NN, il grado di un polinomio utilizzato in una regressione, il tipo di loss function.

Si procede con un approccio a due livelli nel quale per ogni valore **ragionevole** degli iperparametri si esegue l'apprendimento, e al termine della procedura si scelgono gli iperparametri che hanno fornito **prestazioni migliori**.

2.2.1 Valutazione delle prestazioni

Una possibilità consiste nell'utilizzare direttamente la funzione obiettivo per quantificare le prestazioni. In genere però si preferisce una misura legata direttamente alla semantica del problema.

In un problema di **Classificazione** l'accuratezza di classificazione [0...100%] è la percentuale di pattern correttamente classificati. L'errore di classificazione è il complemento.

$$Accuratezza = \frac{\text{pattern correttamente classificati}}{\text{pattern classificati}}$$

$$Errore = 100\% - Accuratezza$$

Attenzione ai problemi di classificazione con cardinalità classi molto sbilanciate: Es. in un problema di classificazione binaria, se una delle due classi è rara, un classificatore "dummy" che non la predice mai potrebbe raggiungere un'accuratezza di classificazione vicina al 100%. Meglio usare Precision/Recall in questo caso.

Nei problemi di **Regressione**, si valuta in genere l' **RMSE (Root Mean Squared Error)** ovvero la radice della media dei quadrati degli scostamenti tra valore vero e valore predetto.

$$RMSE = \sqrt{\frac{1}{N} \sum_{i=1}^N (pred_i - true_i)^2}$$

2.2.2 Matrice di Confusione

La matrice di confusione (confusion matrix) è molto utile nei problemi di **classificazione** (multiclasse) per capire come sono distribuiti gli errori. Nell'esempio un problema di classificazione digit (10 classi).

Sulle righe le classi **true** sulle colonne le classi predicted. Una cella (r,c) riporta la percentuale di casi in cui il sistema ha predetto di classe c un pattern di classe vera r. Idealmente la matrice dovrebbe essere diagonale. Valori elevati (fuori diagonale) indicano concentrazioni di errori.

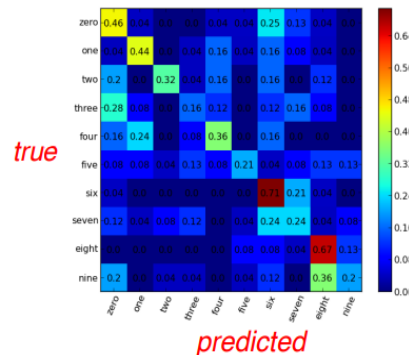


Figure 2

2.2.3 Problemi Closed e Open Set

Nel caso **più semplice** si assume che il pattern da classificare appartenga a una delle classi note (**closed set**). Es. classificare le persone in {uomini, donne}. In molti casi reali invece i pattern da classificare possono appartenere a una delle classi note o a nessuna di queste (**open set**). Es. classificare tutta la frutta in {mele, pere, banane}. Per fare questo ci sono due soluzioni principali: Si aggiunge alle classi un'ulteriore classe fittizia "il resto del mondo" e si aggiungono al training set i cosiddetti "esempi negativi". Si consente al sistema di non assegnare il pattern. A tal fine si definisce una soglia e si assegna il pattern alla classe più probabile solo quando la probabilità è superiore alla soglia.

2.2.4 Classificazione binaria

Consideriamo un problema di classificazione binario, dove le due classi corrispondono a esempi **Positivi** (vera classe) e **Negativi** (classe fittizia resto del mondo). Es. face detection. Positivi: tutte le porzioni di immagine (finestre) in cui appaiono volti. Negativi: finestre (scelte a caso) in cui non compaiono volti. Analogamente possiamo considerare solo la classe positivi e un sistema (con soglia) in grado di calcolare la probabilità p di appartenenza di un pattern alla classe. Sia t il valore della soglia, allora il pattern viene classificato come positivo se $p > t$, come negativo in caso contrario.

Dato un classificatore binario e $T = P + N$ pattern da classificare (P positivi e N negativi), il risultato di ciascuno dei tentativi di classificazione può essere: **True Positive (TP)**: un pattern positivo è stato correttamente assegnato ai positivi, **True Negative (TN)**: un pattern negativo è stato correttamente assegnato ai negativi, **False Positive (FP)**: un pattern negativo è stato erroneamente assegnato ai positivi. Detto anche errore di Tipo I o False. **False Negative (FN)**: un pattern positivo è stato erroneamente assegnato ai negativi. Detto anche errore di Tipo II o Miss.

Passando da errori a frequenze/probabilità:

$$TPR (True Positive Rate) = \frac{TP}{P}$$

$$TNR (True Negative Rate) = \frac{TN}{N}$$

$$FPR (False Positive Rate) = \frac{FP}{N}$$

$$FNR (False Negative Rate) = \frac{FN}{P}$$

Con questa notazione l'accuratezza di classificazione può essere scritta come:

$$Accuracy = \frac{TP + TN}{T = P + N}$$

2.2.5 Sistemi con soglia (DET e ROC)

Nei sistemi con soglia false positive rate e false negative rate sono **funzione** della soglia t . Soglie restrittive (elevate) riducono i false positive a discapito dei false negative; viceversa soglie tolleranti (basse) riducono i false negative a discapito dei false positive.

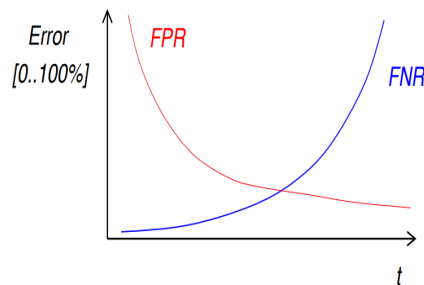


Figure 3

Le due curve possono essere "condensate" in una curva **DET (Detection Error Tradeoff)** che nasconde la soglia. Piuttosto usata è anche la rappresentazione **ROC (Receiver Operating Characteristic)** che in ordinata riporta True positive invece di False negative (ROC è ribaltata verticalmente rispetto a DET).

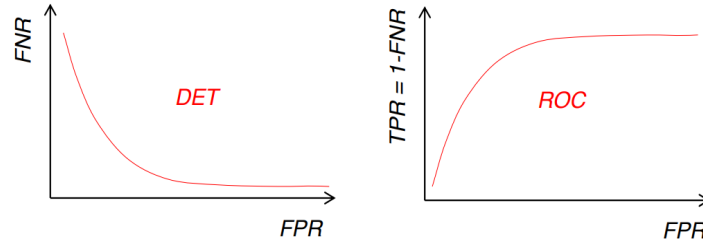
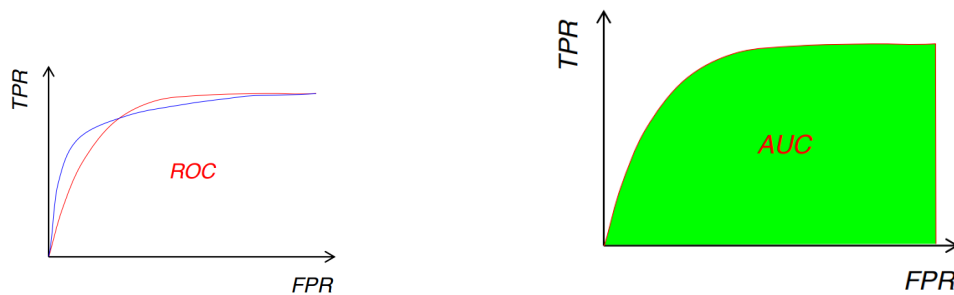


Figure 4

Quale dei due sistemi (rosso, blu) è migliore dell'altro?



Il sistema migliore è quello la cui curve è più "alta", ma in questo caso le curve si intersecano ... e l'ottimalità dipende dal punto di lavoro desiderato. Un confronto può essere fatto "mediando" sui diversi punti di lavoro del sistema. L'**area sotto la curva ROC (AUC)** è uno scalare in $[0,1]$ che caratterizza la prestazione media (maggior è, meglio è). Può essere calcolata come integrale numerico.

Un tipico esempio di sistemi con soglia sono i **sistemi biometrici** di verifica di identità: Questa immagine dell'iride è del soggetto Q?

False positive e False negative in questo caso prendono il nome di **False Match** e **False Non-Match**. Nel caso open set multi-classe, nella biometria si parla anche di **identificazione** distinguendola dalla verifica di identità e si definiscono FPIR (False Positive Identification Rate) e FNIR (False Negative Identification Rate).

2.2.6 Precision e Recall

Notazione molto usata in Information Retrieval e in generale nelle applicazioni di detection. **Precision** indica quanto è accurato il sistema. (quanti True sono veri su quanti True ha dato)

Che percentuale di documenti selezionati è pertinente?

$$Precision = \frac{TP}{TP + FP}$$

Recall quanto è selettivo. (Quanti True su quelli che sono tutti i true possibili True positive e False Negative)

Di tutti i documenti pertinenti quanti ne sono stati selezionati?

$$Recall = \frac{TP}{TP + FN} = \frac{TP}{P} = TPR$$

Un'altra rappresentazione grafica utile per comprendere Precision e Recall è la matrice di confusione 2x2 che evidenzia **TN**, **TP**, **FP**, **FN**: Il classificatore (binario) dell'esempio sotto ha come classe positiva il digit 5 e come classe negativa tutti gli altri digit.

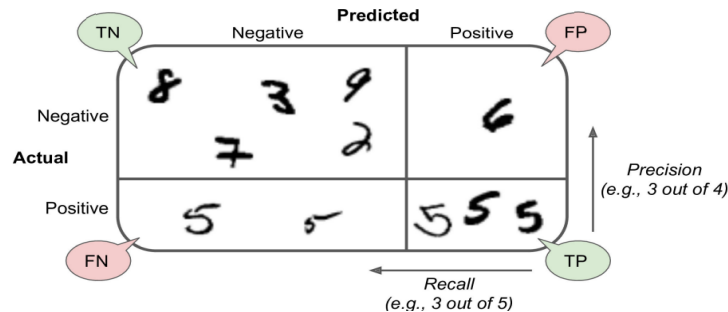


Figure 6

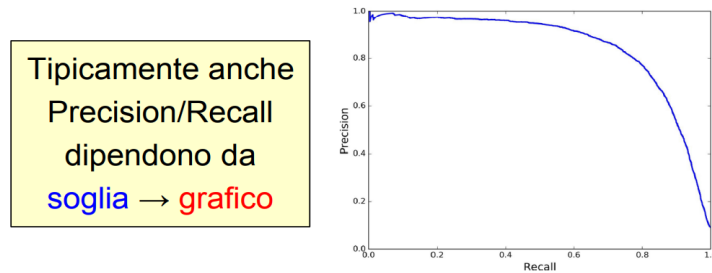


Figure 7

Ogni punto della curva corrisponde a una soglia diversa e la sua posizione corrisponde alla Precision e alla Recall che si ottengono quando si sceglie quella soglia.

2.2.7 Sensitivity e Specificity

In diagnostica medica si usano i termini Sensitivity e Specificity per caratterizzare la bontà di un test (es. tampone Covid-19). Sensitivity=Recall (Tra tutti i sottoposti al test quanti infetti sono correttamente identificati come tali?) Specificity indica quanto è accurato il sistema. (Che percentuale di non-infetti sottoposti al test sono stati dichiarati sani?)

2.2.8 Object Detection: AP e mAP

Detection: per ogni oggetto presente determina la probabilità di appartenenza alla classe più probabile e la corrispondente bounding box (una certa area). Ci possono essere diversi tipi di errori tra cui: oggetto non trovato, classe errata, bounding box sbagliata o imprecisa. Per rendere i sistemi confrontabili, si usa:

Average Precision (AP): è una sorta di AUC calcolata su grafico Recall/Precision per oggetti di una singola classe su tutto il database **medium Average Precision (mAP):** è la media di AP su tutte le classi.

Per calcolare TP, FP si considera prediction corretta quando la classe è giusta e la Intersection over Union (IoU) delle due bounding box (rilevata e vera) è maggiore di un valore dato.

3 Training, Validation, Test, Domanda: Come configurare un Data Set

Il **Training Set** (Train) è l'insieme di pattern su cui addestrare il sistema, trovando il valore ottimo per i parametri θ . Serve a trovare i parametri del sistema.

Il **Validation Set** (Valid) è l'insieme di pattern su cui tarare gli iperparametri H (ciclo esterno). Individuo gli iperparametri Il **Test Set** (Test) è l'insieme di pattern su cui valutare le prestazioni finali del sistema. Sempre forte è la tentazione di tarare gli iperparametri sul test set (overfitting), va evitato.

3.0.1 K-fold Cross-Validation

Una scelta più robusta degli **iperparametri** si ottiene con la procedura di k-fold Cross-Validation. Nell'ipotesi 15 000 **pattern** totali e $k=5$: 5000 pattern sono **scorporati a priori** per il Test, i 10 000 pattern rimanenti sono suddivisi in 5 gruppi (fold) da 2000 pattern ciascuno.

Per ogni combinazione di **iperparametri** H_i che si vuole valutare:

Si esegue 5 volte il training scegliendo uno dei fold come Valid e i 4 rimanenti come Train

Si calcola l'accuratezza avg_acc_i come media/mediana delle 5 accuratezze sui rispettivi Valid,

Si sceglie la combinazione di iperparametri con migliore avg_acc

Scelti gli iperparametri ottimali **si riaddestra il modello su tutto il training set** (5 fold) e, solo a questo punto, si verificano le prestazioni sul **test set**.

3.0.2 Partizionare i Dati

Partizionare i dati in **Training, Validation e Test**, richiede in genere di generare (random) gli indici degli elementi da assegnare ai diversi insiemi per poi procedere alla suddivisione facendo attenzione a dividere nello stesso modo anche le etichette (in classificazione) o valori target (nella regressione). La funzione **train_test_split** di **Scikit Learn** automatizza, questa fase, a partire dalle proporzioni specificate in input, fornendo anche supporto per **shuffling** (ordinamento casuale dei pattern) e **stratification** (selezione bilanciata rispetto a certe categorie o insiemi di valori).

Relativamente a **Cross Validation Scikit Learn** mette a disposizione la funzione $cross_val_score$ che a partire da un unico training set e il numero k di fold, valuta k volte il sistema e ritorna un array di K score.

3.0.3 Selezione Automatica Iperparametri

La ricerca di valori ottimali per gli iperparametri può essere lunga e noiosa. Automatizzare quando possibile:

Grid Search: per ogni iperparametro si definisce un insieme di valori da provare. Il sistema è valutato su tutte le combinazioni di valori di tutti gli iperparametri.

Random Search: sorteggia casualmente valori di iperparametri dai range/distribuzioni specificati/e, eseguendo un numero prefissato di iterazioni.

3.0.4 Convergenza

Il primo obiettivo da perseguire durante l'addestramento è la **convergenza** sul Train set. Consideriamo un classificatore il cui addestramento prevede un processo **iterativo**. Si ha convergenza quando: La **Loss** (output della loss function) ha andamento **decescente**, l'**accuratezza** ha un andamento **crescente**.

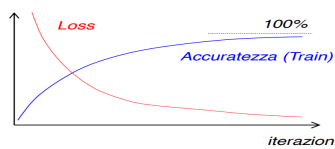


Figure 8

Se la **loss non decresce** (o **oscilla** significativamente) il sistema non converge: il metodo di ottimizzazione non è efficace, gli iperparametri sono fuori range, il learning rate è inadeguato, ci sono errori di implementazione, ecc...

Se la **loss decresce ma l'accuratezza non cresce** probabilmente è stata scelta una loss-function errata.

Se l'**accuratezza non si avvicina al 100% sul Train**, i gradi di libertà del classificatore non sono sufficienti per gestire la complessità del problema.

3.0.5 Generalizzazione e Overfitting

Per **generalizzazione** intendiamo la capacità di **trasferire** l'elevata accuratezza raggiunta su Train a Valid. Se i gradi di libertà del classificatore sono eccessivi, si raggiunge elevata accuratezza su Train, ma non su Valid (scarsa generalizzazione). In questo caso si parla di **overfitting** di Train. Questa situazione si verifica molto facilmente quando Train è di piccole dimensioni. Nei processi di addestramento iterativo, tipicamente dopo un certo numero di iterazioni l'accuratezza su Valid non aumenta più (e può iniziare a decrescere) a causa dell'overfitting. Monitorando l'andamento si può **arrestare l'addestramento** nel punto ideale (**early stopping**).

3.0.6 Controllare l'Overfitting per massimizzare la Generalizzazione

I gradi di libertà del classificatore non devono essere eccessivi, ma adeguati rispetto alla complessità del problema. Buona norma partire **con pochi gradi di libertà** (controllabili attraverso iperparametri) e **via via aumentarli** monitorando accuratezza su Train e Valid. I gradi di libertà influenzano la **regolarità** della soluzione appresa. La regolarità della soluzione può essere talvolta controllata aggiungendo un **fattore regolarizzante** alla loss-function che penalizza soluzioni irregolari. Es. in una rete neurale si possono favorire soluzioni prive di pesi con valori elevati (L_2 **regularization**), oppure solo una parte dei pesi sono diversi da 0 (L_1 **regularization** – **> sparsity**)

3.0.7 In pratica

Non utilizzate approcci di Machine Learning per problemi sui quali **non avete a disposizione sufficienti esempi** per il Training e il Test. Ci sono servizi che etichettano e collezionano dati al posto nostro. I pattern da collezionare devono essere **rappresentativi** del problema che vogliamo risolvere e vanno distribuiti adeguatamente. **Automatizzare con le librerie** le procedure di valutazione delle prestazioni. Scrivere **codice** strutturato ordinato, eseguite debug incrementale e unit testing.

4 Regressione

4.1 Introduzione

I modelli a regressione vengono utilizzati per prevedere variabili target su **scala continua**, il che li rende interessanti per risolvere molte questioni in ambito scientifico e anche industriale, come ad esempio: trovare relazioni fra variabili, valutare tendenze, effettuare previsioni. L'obiettivo di un modello a Regressione Lineare Semplice (univariata) consiste nell'individuare le relazioni esistenti tra un'unica caratteristica (la variabile descrittiva x) e una risposta continua (variabile target y).

4.1.1 Esempio (previsione del prezzo di un andamento)

Consideriamo il caso della previsione del prezzo di un appartamento (variabile target y) data la sua metratura (variabile descrittiva x). Tipicamente in casi come questo abbiamo a disposizione un certo numero di esempi (osservazioni), costituiti da appartamenti già venduti per ciascuno dei quali abbiamo a disposizione l'area in mq o in sq.ft. (x) e il prezzo pagato per l'acquisto (y). Ciascuna delle suddette osservazioni può essere rappresentata da un punto in un piano cartesiano x - y .

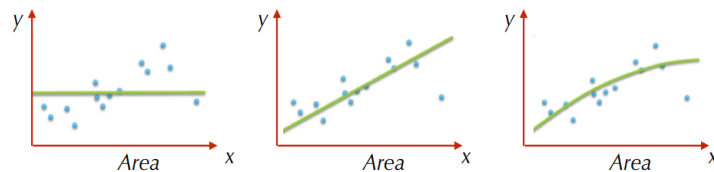


Figure 9

Il problema da risolvere è il seguente: scegliere lo spazio delle ipotesi $\mathcal{H}$ e la funzione $f(x)$ (ipotesi) che approssima meglio le osservazioni disponibili di una funzione sconosciuta, da utilizzare per prevedere i prezzi di altri appartamenti.

4.2 Simple Linear Regression Model

In figura è rappresentato un modello lineare per $f(x)$, dove w_0 rappresenta l'intercetta (punto in cui una retta o una curva incontra uno degli assi cartesiani) e il peso w_1 rappresenta la pendenza della retta. Si noti l'offset verticale che costituisce l'errore che in genere esiste tra la previsione e il valore effettivo. Abbiamo dunque, il valore vero e quello previsto per un certo valore dell'ascissa.

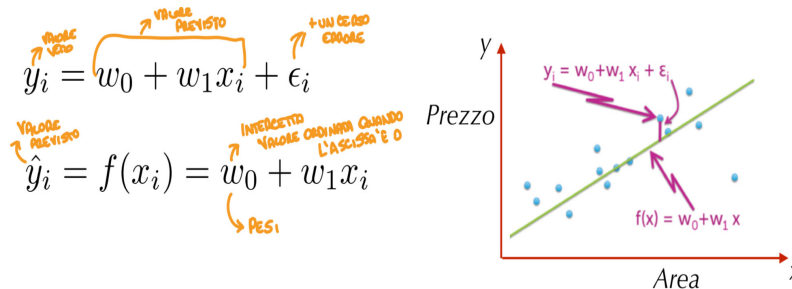


Figure 10

Una volta definito tale modello, ossia la forma della funzione $f(x)$, occorre determinare i due pesi incogniti, ossia l'intercetta e la pendenza, che definiscano la $f(x)$ "migliore" secondo un certo criterio. Un criterio possibile è quello di minimizzare gli errori che si hanno sulle osservazioni.

4.2.1 Residual Sum of Squares (RSS)

Una delle funzioni utilizzate a tal fine, che deve essere per l'appunto minimizzata, è la **Residual Sum of Squares (RSS)**, definita come segue, a partire da N osservazioni disponibili:

$$RSS(w_0, w_1) = \sum_{i=1}^N (y_i - \hat{y}_i)^2 = \sum_{i=1}^N [y_i - (w_0 + w_1 x_i)]^2$$

$\xrightarrow{\text{SOMA DEI QUADRATI DEGLI ERRORI}} N \rightarrow \text{NUM. OSSERVAZIONI}$
 $\xrightarrow{\text{ERRORE}}$

Figure 11

Il problema di addestrare il nostro modello è dunque quello di trovare i valori dei due pesi $\hat{w}_0$ e $\hat{w}_1$ che minimizzano la funzione **RSS**. In sintesi, il processo di apprendimento è formulato come una ricerca di ottimizzazione (ricerca

del minimo) nello **spazio dei pesi**. A tal fine possiamo calcolare e avvalerci del **gradiente** della "misura d'errore" **RSS** definita in precedenza. Si può dimostrare che la **RSS** è una funzione convessa. Per funzioni convesse, quando il gradiente è uguale a zero si ha un minimo globale.

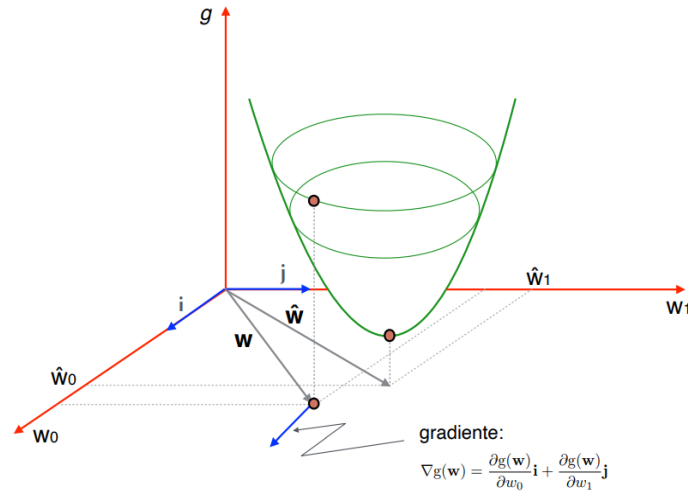


Figure 12

Quindi, per ottimizzare mi sposto verso il gradiente, per minimizzare mi sposto verso il lato opposto.

4.2.2 Il processo di Training

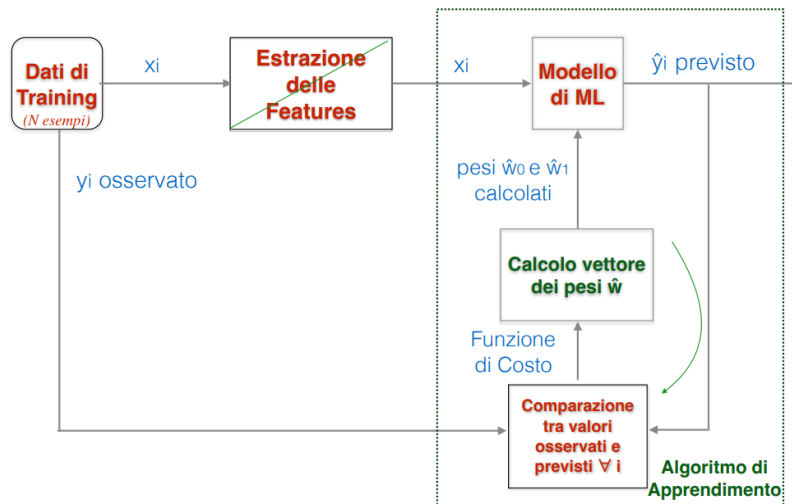


Figure 13

Dati di training: insieme di esempi (x_i, y_i) relativi a casi conosciuti (i punti nel piano x-y), da utilizzare per calcolare la funzione di costo RSS.

Estrazione di features: in questo caso tale funzione è inattiva, nel senso che riproduce in uscita il suo ingresso x_i .

Modello di ML: ipotesi f scelta, istanziata con i valori dei pesi più opportuni.

Comparazione tra dati osservati e dati previsti: per ogni esempio abbiamo il valore vero y_i e il valore previsto

$\hat{y}_i$, da utilizzare per il calcolo della funzione RSS.

Algoritmo di Apprendimento: algoritmo che calcola i pesi che minimizzano la funzione di costo RSS, da utilizzare per definire il modello di ML.

4.2.3 Gradiente della funzione RSS

Il gradiente della funzione RSS:

$$\text{RSS}(w_0, w_1) = \sum_{i=1}^N [y_i - (w_0 + w_1 x_i)]^2$$

Figure 14

è definito come segue:

$$\nabla \text{RSS}(w_0, w_1) = \begin{bmatrix} \frac{\partial \text{RSS}}{\partial w_0} \\ \frac{\partial \text{RSS}}{\partial w_1} \end{bmatrix}$$

Figure 15

$$\begin{aligned} \frac{\partial \text{RSS}}{\partial w_0} &= \sum_{i=1}^N 2[y_i - (w_0 + w_1 x_i)]^1 \cdot (-1) = -2 \sum_{i=1}^N [y_i - (w_0 + w_1 x_i)] \\ \frac{\partial \text{RSS}}{\partial w_1} &= \sum_{i=1}^N 2[y_i - (w_0 + w_1 x_i)]^1 \cdot (-x_i) = -2 \sum_{i=1}^N [y_i - (w_0 + w_1 x_i)] x_i \end{aligned}$$

$$\nabla \text{RSS}(w_0, w_1) = \begin{bmatrix} -2 \sum_{i=1}^N [y_i - (w_0 + w_1 x_i)] \\ -2 \sum_{i=1}^N [y_i - (w_0 + w_1 x_i)] x_i \end{bmatrix}$$

Figure 16

Una volta calcolato il gradiente della funzione **RSS**, ci sono due possibili approcci per minimizzare la funzione di costo: **"Forma chiusa"**, si eguaglia il gradiente a 0 (ossia al vettore nullo) e si risolvono le equazioni (non sempre è possibile o conveniente dal punto di vista computazionale). Algoritmo di Discesa del Gradiente (Hill climbing al contrario) (**Gradient Descent**) (richiede la definizione del criterio di convergenza e dello "step size"). Si parte da un punto dello spazio dei pesi, l'algoritmo si ferma quando arriva a un punto sufficientemente vicino al minimo assoluto.

4.2.4 Forma Chiusa

Poniamo il gradiente uguale al vettore nullo:

$$\nabla \text{RSS}(w_0, w_1) = \begin{bmatrix} -2 \sum_{i=1}^N [y_i - (w_0 + w_1 x_i)] \\ -2 \sum_{i=1}^N [y_i - (w_0 + w_1 x_i)] x_i \end{bmatrix} = \mathbf{0}$$

ossia:

$$\begin{aligned} -2 \sum_{i=1}^N [y_i - (w_0 + w_1 x_i)] &= 0 \\ -2 \sum_{i=1}^N [y_i - (w_0 + w_1 x_i)] x_i &= 0 \end{aligned}$$

Dalla seconda equazione otteniamo:

$$\sum_{i=1}^N x_i y_i - \hat{w}_0 \sum_{i=1}^N x_i - \hat{w}_1 \sum_{i=1}^N x_i^2 = 0$$

da cui si ha:

$$\hat{w}_1 = \frac{\sum_{i=1}^N x_i y_i - \hat{w}_0 \sum_{i=1}^N x_i}{\sum_{i=1}^N x_i^2}$$

Dalla prima equazione otteniamo:

$$\sum_{i=1}^N y_i - \hat{w}_0 \sum_{i=1}^N 1 - \hat{w}_1 \sum_{i=1}^N x_i = 0$$

da cui si ha:

$$\hat{w}_0 = \frac{\sum_{i=1}^N y_i}{N} - \hat{w}_1 \frac{\sum_{i=1}^N x_i}{N}$$

e con facili passaggi otteniamo:

$$\hat{w}_1 = \frac{\sum_{i=1}^N x_i y_i - \frac{\sum_{i=1}^N x_i \sum_{i=1}^N y_i}{N}}{\sum_{i=1}^N x_i^2 - \frac{(\sum_{i=1}^N x_i)^2}{N}}$$

che, insieme a:

$$\hat{w}_0 = \frac{\sum_{i=1}^N y_i}{N} - \hat{w}_1 \frac{\sum_{i=1}^N x_i}{N}$$

vista in precedenza, ci consente di calcolare i valori dei due pesi che minimizzano la funzione RSS.

4.2.5 Gradient Descent

Con questo approccio dobbiamo aggiornare i pesi in modo tale da spostarci nella direzione opposta al gradiente

$$\mathbf{w}^{(t+1)} \leftarrow \mathbf{w}^{(t)} - \alpha \cdot \nabla \text{RSS}(\mathbf{w}^{(t)})$$

dove: $\mathbf{w} = \begin{bmatrix} w_0 \\ w_1 \end{bmatrix}$

ossia:

$$\begin{aligned} w_0^{(t+1)} &\leftarrow w_0^{(t)} - \alpha \cdot \frac{\partial \text{RSS}(\mathbf{w}^{(t)})}{\partial w_0} \\ w_1^{(t+1)} &\leftarrow w_1^{(t)} - \alpha \cdot \frac{\partial \text{RSS}(\mathbf{w}^{(t)})}{\partial w_1} \end{aligned}$$

Rivediamo l'espressione del gradiente di RSS:

$$\nabla \text{RSS}(w_0, w_1) = \begin{bmatrix} -2 \sum_{i=1}^N [y_i - (w_0 + w_1 x_i)] \\ -2 \sum_{i=1}^N [y_i - (w_0 + w_1 x_i)] x_i \end{bmatrix} = \begin{bmatrix} -2 \sum_{i=1}^N [y_i - \hat{y}_i(w_0, w_1)] \\ -2 \sum_{i=1}^N [y_i - \hat{y}_i(w_0, w_1)] x_i \end{bmatrix}$$

L'aggiornamento dei due pesi può dunque essere effettuato come segue, scegliendo un opportuno **step size**:

$$\begin{aligned} w_0^{(t+1)} &\leftarrow w_0^{(t)} + 2\alpha \cdot \sum_{i=1}^N [y_i - \hat{y}_i(\mathbf{w}^{(t)})] \\ w_1^{(t+1)} &\leftarrow w_1^{(t)} + 2\alpha \cdot \sum_{i=1}^N [y_i - \hat{y}_i(\mathbf{w}^{(t)})] x_i \end{aligned}$$

Dobbiamo infine scegliere un **criterio di convergenza**. Come già detto, per funzioni convesse si ha un minimo

globale quando il gradiente è uguale a zero. In pratica possiamo fermarci quando:

$$\|\nabla \text{RSS}(\mathbf{w}^{(t)})\|_2 \leq \epsilon$$

Figure 20

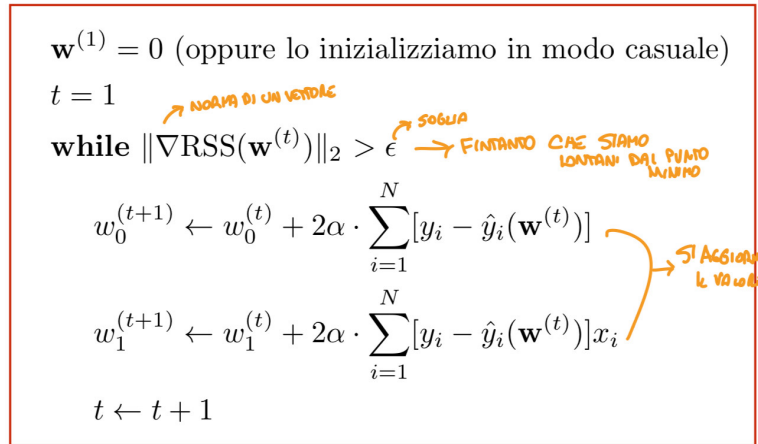


Figure 21

4.3 Multiple Regression (Linear Regression con Multiple Features)

Fino ad ora abbiamo ipotizzato, per la funzione $f(x)$, un andamento lineare per il nostro caso di studio relativo ai prezzi degli appartamenti. Tuttavia l'esperienza comune ci induce a pensare che la relazione tra le due variabili (area e prezzo di un appartamento) non sia proprio lineare. In genere, all'aumentare della metratura il prezzo aumenta ma non in modo esattamente proporzionale. Potremmo ipotizzare ad esempio una funzione quadratica o addirittura polinomiale di grado p

$$f(x) = w_0 + w_1x + w_2x^2$$

$$f(x) = w_0 + w_1x + w_2x^2 + \dots + w_px^p$$

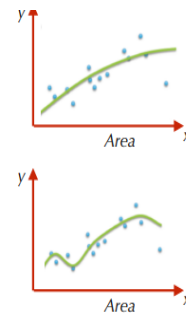


Figure 22

In quest'ultimo caso avremmo una **Polynomial Regression** il cui modello è il seguente:

$$y_i = w_0 + w_1x_i + w_2x_i^2 + \dots + w_px_i^p + \epsilon_i$$

In genere, le potenze della x sono trattate come differenti

features:

$$\begin{aligned} \text{feature 1} &= 1 \\ \text{feature 2} &= x \\ \text{feature 3} &= x^2 \\ &\dots\dots\dots \\ \text{feature } p+1 &= x^p \end{aligned}$$

Il caso generale, con un solo input x_i , è il seguente:

$$\begin{aligned} y_i &= w_0\phi_0(x_i) + w_1\phi_1(x_i) + \dots + w_D\phi_D(x_i) + \epsilon_i = \\ &= \sum_{j=0}^D w_j\phi_j(x_i) + \epsilon_i \end{aligned}$$

dove le features che compaiono possono assumere forme diverse (non necessariamente solo potenze della x).

Se volessimo considerare anche altre caratteristiche in input avremmo un vettore x_i per ogni esempio noto, le cui componenti sono appunto l'area, i bagni ecc...

Anche in questo caso possiamo utilizzare, come funzione di costo da minimizzare, la **RSS** definita come segue, a partire da N osservazioni disponibili:

$$\text{RSS}(\mathbf{w}) = \sum_{i=1}^N (y_i - \hat{y}_i)^2 = \sum_{i=1}^N [y_i - (w_0\phi_0(\mathbf{x}_i) + \dots + w_D\phi_D(\mathbf{x}_i))]^2$$

Figure 24

Il problema di addestrare il nostro modello è dunque quello di trovare i valori dei pesi $\hat{w}_0, \hat{w}_1, \dots, \hat{w}_D$ che minimizzano la funzione RSS (convessa anche in questo caso)

4.3.1 Il processo di Training

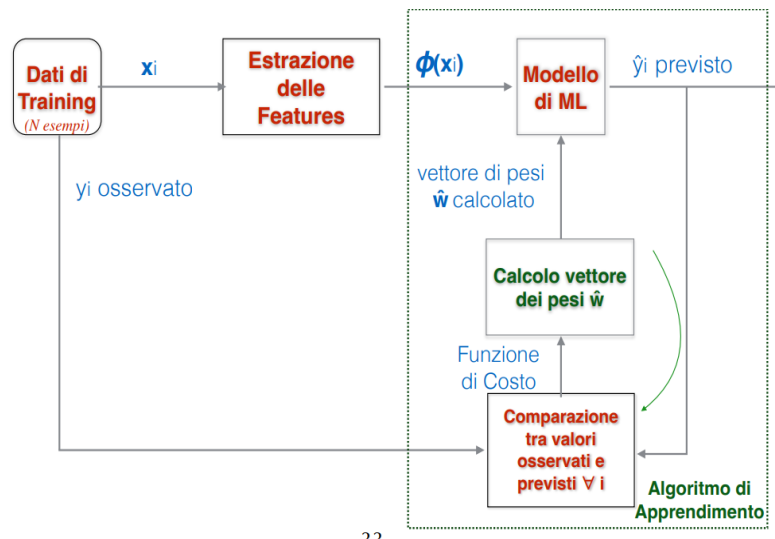


Figure 25

4.4 Regression Model in notazione Matriciale

In molti casi può essere conveniente usare una notazione matriciale. L'espressione:

$$y_i = \sum_{j=0}^D w_j \phi_j(\mathbf{x}_i) + \epsilon_i$$

Figure 26

Può essere riscritta:

$$y_i = [w_0 \ w_1 \ \dots \ w_D] \cdot \begin{bmatrix} \phi_0(\mathbf{x}_i) \\ \phi_1(\mathbf{x}_i) \\ \dots \\ \phi_D(\mathbf{x}_i) \end{bmatrix} + \epsilon_i$$

oppure:

$$y_i = [\phi_0(\mathbf{x}_i) \ \phi_1(\mathbf{x}_i) \ \dots \ \phi_D(\mathbf{x}_i)] \cdot \begin{bmatrix} w_0 \\ w_1 \\ \dots \\ w_D \end{bmatrix} + \epsilon_i$$

In sintesi:

$$y_i = \sum_{j=0}^D w_j \phi_j(\mathbf{x}_i) + \epsilon_i = \mathbf{w}^T \cdot \boldsymbol{\phi}(\mathbf{x}_i) + \epsilon_i = \boldsymbol{\phi}^T(\mathbf{x}_i) \cdot \mathbf{w} + \epsilon_i$$

dove:

$$\mathbf{x}_i = \begin{bmatrix} x_{i,1} \\ x_{i,2} \\ \dots \\ x_{i,d} \end{bmatrix} \quad \mathbf{w} = \begin{bmatrix} w_0 \\ w_1 \\ \dots \\ w_D \end{bmatrix} \quad \boldsymbol{\phi}(\mathbf{x}_i) = \begin{bmatrix} \phi_0(\mathbf{x}_i) \\ \phi_1(\mathbf{x}_i) \\ \dots \\ \phi_D(\mathbf{x}_i) \end{bmatrix}$$

Possiamo infine rappresentare tutte le osservazioni y in modo compatto come segue:

$$\begin{bmatrix} y_1 \\ y_2 \\ \dots \\ y_N \end{bmatrix} = \begin{bmatrix} \phi_0(\mathbf{x}_1) & \phi_1(\mathbf{x}_1) & \dots & \phi_D(\mathbf{x}_1) \\ \phi_0(\mathbf{x}_2) & \phi_1(\mathbf{x}_2) & \dots & \phi_D(\mathbf{x}_2) \\ \dots & \dots & \dots & \dots \\ \phi_0(\mathbf{x}_N) & \phi_1(\mathbf{x}_N) & \dots & \phi_D(\mathbf{x}_N) \end{bmatrix} \cdot \begin{bmatrix} w_0 \\ w_1 \\ \dots \\ w_D \end{bmatrix} + \begin{bmatrix} \epsilon_1 \\ \epsilon_2 \\ \dots \\ \epsilon_N \end{bmatrix}$$

ossia:

$$\mathbf{y} = \boldsymbol{\Phi} \cdot \mathbf{w} + \boldsymbol{\epsilon}$$

dove:

$$\mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \dots \\ y_N \end{bmatrix} \quad \boldsymbol{\epsilon} = \begin{bmatrix} \epsilon_1 \\ \epsilon_2 \\ \dots \\ \epsilon_N \end{bmatrix}$$

$$\boldsymbol{\Phi} = \begin{bmatrix} \phi_0(\mathbf{x}_1) & \phi_1(\mathbf{x}_1) & \dots & \phi_D(\mathbf{x}_1) \\ \phi_0(\mathbf{x}_2) & \phi_1(\mathbf{x}_2) & \dots & \phi_D(\mathbf{x}_2) \\ \dots & \dots & \dots & \dots \\ \phi_0(\mathbf{x}_N) & \phi_1(\mathbf{x}_N) & \dots & \phi_D(\mathbf{x}_N) \end{bmatrix}$$

Calcoliamo ora la funzione RSS:

$$\text{RSS}(\mathbf{w}) = \sum_{i=1}^N (y_i - \hat{y}_i)^2 = \sum_{i=1}^N [y_i - (\phi_0(\mathbf{x}_i)w_0 + \dots + \phi_D(\mathbf{x}_i)w_D)]^2 = \sum_{i=1}^N [y_i - \boldsymbol{\phi}^T(\mathbf{x}_i) \cdot \mathbf{w}]^2$$

che possiamo scrivere anche così:

$$\text{RSS}(\mathbf{w}) = \sum_{i=1}^N (y_i - \hat{y}_i)^2 = \sum_{i=1}^N \epsilon_i^2 = \boldsymbol{\epsilon}^T \cdot \boldsymbol{\epsilon}$$

Da una precedente espressione per $\mathbf{y}$ ricaviamo il vettore $\boldsymbol{\epsilon}$:

$$\mathbf{y} = \boldsymbol{\Phi}\mathbf{w} + \boldsymbol{\epsilon} \quad \Rightarrow \quad \boldsymbol{\epsilon} = \mathbf{y} - \boldsymbol{\Phi}\mathbf{w}$$

La funzione RSS assume pertanto la seguente forma in notazione matriciale:

$$\text{RSS}(\mathbf{w}) = (\mathbf{y} - \boldsymbol{\Phi}\mathbf{w})^T (\mathbf{y} - \boldsymbol{\Phi}\mathbf{w})$$

4.4.1 Gradiente della funzione RSS

Calcoliamo ora il gradiente della funzione RSS, partendo dalla precedente espressione matriciale.

Applicando una nota regola di calcolo differenziale matriciale si ottiene:

$$\nabla \text{RSS}(\mathbf{w}) = \nabla [(\mathbf{y} - \boldsymbol{\Phi}\mathbf{w})^T (\mathbf{y} - \boldsymbol{\Phi}\mathbf{w})] = -2\boldsymbol{\Phi}^T (\mathbf{y} - \boldsymbol{\Phi}\mathbf{w})$$

Figure 30

4.4.2 Forma Chiusa

Poniamo il gradiente uguale al vettore nullo:

$$\nabla \text{RSS}(\mathbf{w}) = -2\mathbf{\Phi}^T(\mathbf{y} - \mathbf{\Phi}\mathbf{w}) = \mathbf{0}$$

$$\begin{aligned} -2\Phi^T \mathbf{y} + 2\Phi^T \Phi \hat{\mathbf{w}} &= 0 \\ \Phi^T \Phi \hat{\mathbf{w}} &= \Phi^T \mathbf{y} \\ (\Phi^T \Phi)^{-1} (\Phi^T \Phi) \hat{\mathbf{w}} &= (\Phi^T \Phi)^{-1} \Phi^T \mathbf{y} \\ \mathbf{I} \hat{\mathbf{w}} &= (\Phi^T \Phi)^{-1} \Phi^T \mathbf{y} \end{aligned}$$

da cui si ha:

$$\hat{\mathbf{w}} = (\Phi^T \Phi)^{-1} \Phi^T \mathbf{y}$$

Figure 31

4.4.3 Gradient Descent

Dobbiamo aggiornare il vettore dei pesi in modo tale da spostarci nella direzione opposta al gradiente:

$$\mathbf{w}^{(t+1)} \leftarrow \mathbf{w}^{(t)} - \alpha \cdot \nabla \text{RSS}(\mathbf{w}^{(t)})$$

dove:

$$\mathbf{w}^{(t)} = \begin{bmatrix} w_0^{(t)} \\ w_1^{(t)} \\ \dots \\ w_D^{(t)} \end{bmatrix} \quad \nabla \text{RSS}(\mathbf{w}^{(t)}) = \begin{bmatrix} \frac{\partial \text{RSS}}{\partial w_0} \\ \frac{\partial \text{RSS}}{\partial w_1} \\ \dots \\ \frac{\partial \text{RSS}}{\partial w_D} \end{bmatrix}$$

I singoli pesi devono dunque essere aggiornati come segue:

$$\begin{aligned} w_0^{(t+1)} &\leftarrow w_0^{(t)} - \alpha \cdot \frac{\partial \text{RSS}(\mathbf{w}^{(t)})}{\partial w_0} \\ w_1^{(t+1)} &\leftarrow w_1^{(t)} - \alpha \cdot \frac{\partial \text{RSS}(\mathbf{w}^{(t)})}{\partial w_1} \\ &\dots\dots\dots \\ w_j^{(t+1)} &\leftarrow w_j^{(t)} - \alpha \cdot \frac{\partial \text{RSS}(\mathbf{w}^{(t)})}{\partial w_j} \\ &\dots\dots\dots \\ w_D^{(t+1)} &\leftarrow w_D^{(t)} - \alpha \cdot \frac{\partial \text{RSS}(\mathbf{w}^{(t)})}{\partial w_D} \end{aligned}$$

Per comprendere gli aggiornamenti da fare per i singoli pesi, calcoliamo la derivata parziale di RSS, espressa in questa forma:

$$\text{RSS}(\mathbf{w}^{(t)}) = \sum_{i=1}^N [y_i - \hat{y}_i(\mathbf{w}^{(t)})]^2 = \sum_{i=1}^N \{y_i - [\phi_0(\mathbf{x}_i)w_0^{(t)} + \dots + \phi_j(\mathbf{x}_i)w_j^{(t)} + \dots + \phi_D(\mathbf{x}_i)w_D^{(t)}]\}^2$$

rispetto al generico peso j-esimo:

$$\begin{aligned} \frac{\partial \text{RSS}(\mathbf{w}^{(t)})}{\partial w_j} &= \sum_{i=1}^N 2[y_i - \hat{y}_i(\mathbf{w}^{(t)})] \cdot \left[-\frac{\partial \hat{y}_i(\mathbf{w}^{(t)})}{\partial w_j}\right] = \\ &= 2 \sum_{i=1}^N [y_i - \hat{y}_i(\mathbf{w}^{(t)})] \cdot [-\phi_j(\mathbf{x}_i)] = -2 \sum_{i=1}^N \phi_j(\mathbf{x}_i) [y_i - \hat{y}_i(\mathbf{w}^{(t)})] \end{aligned}$$

Figure 33

Anche in questo caso dobbiamo scegliere un criterio di convergenza, quello scelto per il caso singolo va bene.

```

 $\mathbf{w}^{(1)} = 0$  (oppure lo inizializziamo in modo casuale)
 $t = 1$ 
while  $\|\nabla \text{RSS}(\mathbf{w}^{(t)})\|_2 > \epsilon$ 
  for  $j = 0, 1, \dots, D$ 
    derivata_parziale[j] =  $-2 \sum_{i=1}^N \phi_j(\mathbf{x}_i) [y_i - \hat{y}_i(\mathbf{w}^{(t)})]$ 
     $w_j^{(t+1)} \leftarrow w_j^{(t)} - \alpha * \text{derivata\_parziale}[j]$ 
   $t \leftarrow t + 1$ 

```

Figure 34

4.5 Workflow per Regression

I due task importanti che dobbiamo attuare nella regression sono: scelta del modello di regressione (**model selection**), valutazione del modello (**model assessment**).

Nel **Model selection** scegliamo dei parametri di tuning λ che controllano la complessità del modello (es. grado del polinomio). Nel **Model assessment** effettuiamo la valutazione del Generalization Error (o True Error)

Ovviamente è importante evitare il peaking, ovvero non bisogna selezionare le ipotesi in base alle sue prestazioni sull'insieme di test e l'overfitting, ovvero strutturare tutto sulla base dei dati in possesso.

Un approccio **ingenuo** al problema potrebbe essere il seguente:

1. Model selection

- Per ogni modello di complessità λ :
 - stima dei parametri $\hat{w}_\lambda$ sui training data
 - valutazione delle prestazioni sui test data
 - scelta del parametro λ (λ^*) che comporta il più basso test error

2. Model assessment

- Considerare il test error calcolato su $\hat{w}_{\lambda^*}$ (fitted model per la complessità scelta λ^*) per approssimare il True Error.

Figure 35

L'approccio in questo caso è il seguente:

1. Model selection

- Per ogni modello di complessità λ :
 - stima dei parametri $\hat{w}_\lambda$ sul training set
 - valutazione delle prestazioni sul validation set
 - scelta del parametro λ (λ^*) che comporta il più basso errore sul validation set

2. Model assessment

- Calcolo del test error (usando dunque il test set) con $\hat{w}_{\lambda^*}$ per approssimare il True Error.

Figure 36

Tipici split sono i seguenti:

<i>Training Set</i>	<i>Validation Set</i>	<i>Test Set</i>
80%	10%	10%
50%	25%	25%

Figure 37

5 Regressione Errori

5.1 Valutazione delle Prestazioni

La valutazione degli errori delle prestazioni è di importanza cruciale per poter apprezzare il metodo che stiamo utilizzando per le nostre previsioni. Essa ci aiuta nella scelta tra modelli di diversa complessità a nostra disposizione. A tal fine occorre definire una metrica che ci consenta di valutare quanto perdiamo (loss) quando facciamo una certa previsione.

5.2 Loss Function

Possiamo indicare una funzione di Loss come segue:

$$L[y, f_{\hat{\mathbf{w}}}(\mathbf{x})]$$

Figure 38

Dove y è l'actual value, mentre la funzione f ci fornisce il valore previsto $\hat{y}$. Tale funzione L rappresenta il costo che abbiamo se usiamo la f con il vettore dei pesi $\hat{w}$ a fronte dell'input $\mathbf{x}$. In pratica la Loss Function quantifica quanto è lontana la previsione del modello dai valori reali. La Loss Function viene utilizzata durante il training per aggiornare i pesi $\hat{w}$ della funzione f . Lo scopo è minimizzare la funzione di perdita, così il modello fornirà previsioni migliori.

5.2.1 Esempi di Loss Function

La funzione di Loss può essere ad esempio definita come Errore Assoluto (Absolute Error):

$$L[y, f_{\hat{w}}(\mathbf{x})] = |y - f_{\hat{w}}(\mathbf{x})|$$

oppure come Errore Quadratico (Squared Error):

$$L[y, f_{\hat{w}}(\mathbf{x})] = [y - f_{\hat{w}}(\mathbf{x})]^2$$

Tali esempi di funzione assumono che il "loss" per underpredicting sia uguale a quello di overpredicting.

Figure 39

5.3 Le tre misure di Loss

5.3.1 Training Error

Il **training error** è la media degli errori calcolati con la funzione di perdita (L) sui dati del training set. Consideriamo ancora l'esempio relativo ai prezzi degli appartamenti. Supponiamo di avere a disposizione le osservazioni come rappresentate in figura (puntini blu). Dobbiamo decidere un modello da utilizzare e scegliere un sotto insieme delle osservazioni per effettuare la fase di training del modello. Possiamo ad esempio scegliere un modello quadratico e calcolare i pesi w tali da minimizzare la funzione RSS

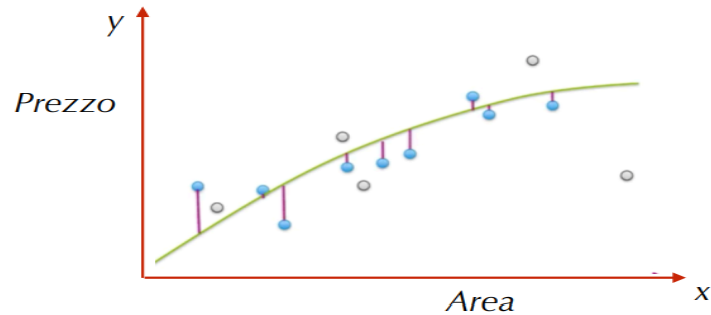


Figure 40

Una volta stimati i parametri del modello, possiamo valutare il training error di tale modello stimato come segue: Definizione di una Loss Function e Calcolo del Training Error come "Average Loss", definito sugli N punti di training:

$$\text{Training Error} = \frac{1}{N} \cdot \sum_{i=1}^N L[y_i, f_{\hat{w}}(\mathbf{x}_i)]$$

Figure 41

Più è complesso il modello e minore sarà l'errore. Il Training Error in generale non è una buona misura dato che è eccessivamente ottimistico, proprio perchè il vettore $\hat{w}$ è calcolato affinché il modello si adatti ai dati di training.

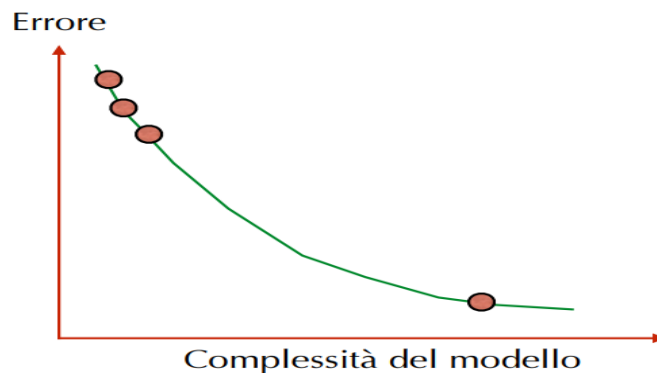
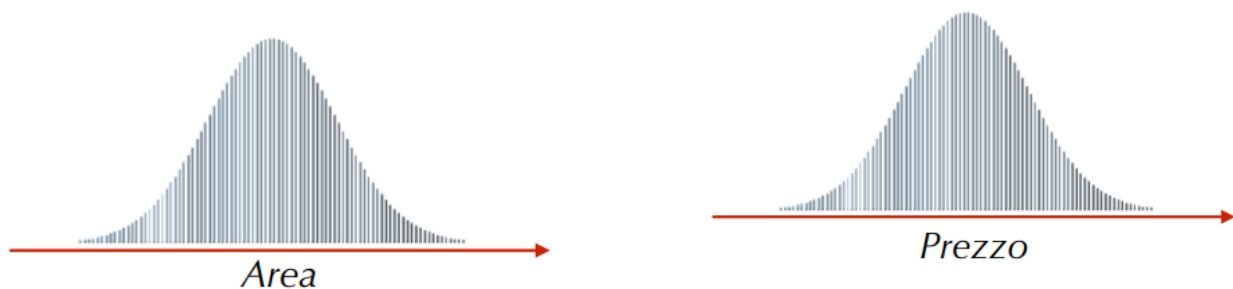


Figure 42

5.3.2 Generalization Error

Il generalization error (o errore di generalizzazione) misura quanto bene un modello addestrato riesce a fare previsioni accurate su dati nuovi e mai visti prima, cioè al di fuori del training set. Sarebbe interessante conoscere il "loss" prendendo in considerazione tutte le possibili coppie (x, y) ossia, per il nostro esempio degli appartamenti, tutte le possibili case della zona presa in considerazione, calcolando la media della funzione loss su tali appartamenti. In genere però nel nostro data set abbiamo soltanto un numero limitato di osservazioni (x, y) . Per effettuare una stima di tale costo dovremmo cercare di pesare le varie coppie (x, y) in base alla probabilità che hanno di essere presenti nella zona d'interesse. Dunque è utile prendere in considerazione la distribuzione degli appartamenti in base al valore della loro area. Potremmo inoltre considerare la distribuzione delle case in base al loro prezzo, a parità di area.



$$\text{Generalization Error} = E_{\mathbf{x}, y}[L(y, f_{\hat{w}}(\mathbf{x}))]$$

Figure 44

Lo definiamo come l'average value della funzione Loss, calcolato su tutte le possibili coppie (x, y) pesate in base alla loro probabilità di comparire nella zona.

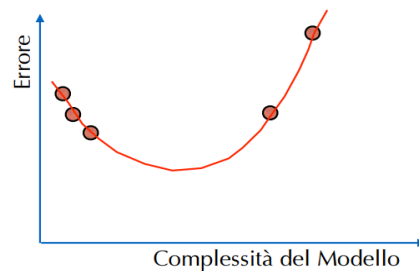


Figure 45

C'è da ricordare che il Generalization Error non è calcolabile, per calcolarlo dovremmo conoscere la "true distribution" delle probabilità vista prima, cosa che non sappiamo fare.

5.3.3 Test Error

Il Test Error (o errore di test) è una misura dell'accuratezza di un modello su un test set, cioè un insieme di dati separato e mai usato durante l'addestramento o la validazione. È un'approssimazione empirica dell'errore di generalizzazione. Definito come average loss sui punti dell'insieme di test:

$$\text{Test Error} = \frac{1}{N_{\text{test}}} \cdot \sum_{i \in \text{test}} L[y_i, f_{\hat{w}}(\mathbf{x}_i)]$$

Figure 46

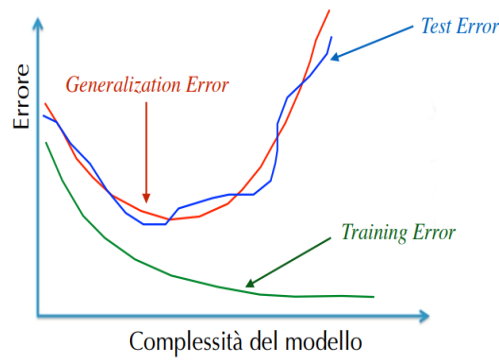


Figure 47

5.4 Overfitting

Dato un modello con parametri $\hat{w}$ si ha overfitting se esiste un modello con i parametri stimati w' tale che:

1. $\text{training error}(\hat{w}) < \text{training error}(w')$
2. $\text{true error}(\hat{w}) > \text{true error}(w')$

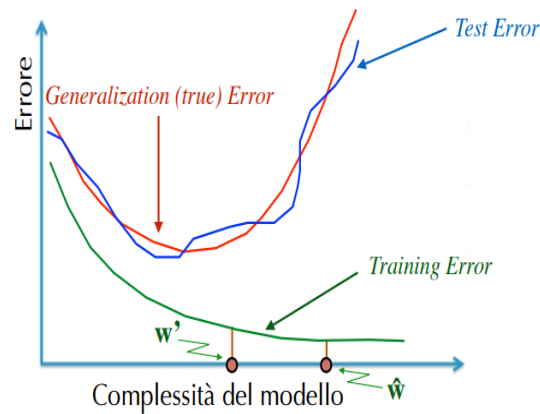


Figure 48

6 Classificatore di Bayes

6.1 Introduzione

Quando un agente conosce l'ambiente circostante, l'approccio logico gli consente di derivare piani efficaci. Ma gli agenti non hanno quasi mai accesso a tutta l'informazione necessaria: devono quindi agire in condizioni di **incertezza**. La **teoria della probabilità** è il modo migliore di ragionare in condizioni di incertezza.

6.2 Approccio Bayesiano

Il problema è posto in **termini probabilistici**. Se tutte le **distribuzioni in gioco sono note** l'**approccio Bayesiano** costituisce la migliore regola di classificazione possibile: **soluzione OTTIMA!**.

Sia V uno spazio di pattern d -dimensionali e $W = \{w_1, w_2, \dots, w_s\}$ un insieme di s classi disgiunte costituite da elementi di V .

Per ogni $x \in V$ e per ogni $w_i \in W$, indichiamo con $p(x|w_i)$ la **densità di probabilità condizionale** di x data w_i , ovvero la densità di probabilità che il prossimo pattern sia x sotto l'ipotesi che la sua classe di appartenenza sia w_i . (Per un nuovo pattern x da classificare: si contano le occorrenze dei pattern del training set delle due classi in un intorno di x (vedi es maschi femmine) $(\frac{numPattAppartendentiAllIntorno}{numPattAppartendentiAllaClasse})$)

Per ogni $w_i \in W$, indichiamo con $P(w_i)$ la **probabilità a priori** di w_i ovvero la probabilità, indipendentemente dall'osservazione, che il prossimo pattern da classificare sia di classe w_i . (Si considera l'occorrenza dei pattern nel training set $\frac{numPattAppartendentiAllaClasse}{numPattTotali}$)

Per ogni $x \in V$ indichiamo con $p(x)$ la **densità di probabilità assoluta** di x , ovvero la densità di probabilità che il prossimo pattern da classificare sia x

$$p(x) = \sum_{i=1}^s p(x|w_i) \cdot P(w_i) \quad \text{dove} \quad \sum_{i=1}^s P(w_i) = 1$$

Figure 49

Per ogni $w_i \in W$ e per ogni $x \in V$ indichiamo con $P(w_i|x)$ la **probabilità a posteriori** di w_i dato x , ovvero la probabilità che avendo osservato il pattern x , la classe di appartenenza sia w_i . Per il **teorema di Bayes**:

$$P(w_i|x) = \frac{p(x|w_i) \cdot P(w_i)}{p(x)}$$

6.3 Classificatore di Bayes

Dato un pattern x da classificare in una delle s classi $w_1, w_2, \dots, w_s$ di cui sono note: le **probabilità a priori** $P(w_1), \dots$ e le **densità di probabilità condizionali** $p(x|w_1) \dots$

la **regola di classificazione** di Bayes assegna x alla classe b per cui è massima la probabilità a posteriori:

$$b = \operatorname{argmax}_{i=1..s} \{P(w_i|x)\}$$

Massimizzare la probabilità a posteriori significa **massimizzare** la densità di probabilità condizionale tenendo comunque conto della probabilità a priori delle classi. (Per capire meglio guarda esempio maschi/femmine sulle slide)

6.3.1 Esempio maschi femmine

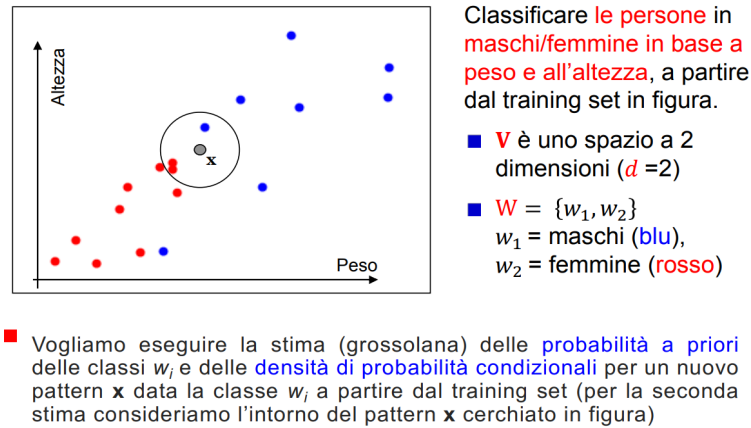


Figure 50

Una volta fatta la stima applichiamo il **teorema di Bayes** e facciamo la probabilità a posteriori. L'approccio Bayesiano assegnerà il pattern alla classe con probabilità a posteriori più alta (femmine).

6.4 Approcci parametrici e non

Mentre la stima delle probabilità a priori è abbastanza semplice la **conoscenza** delle densità condizionali è possibile "solo in teoria"; nella **pratica** due soluzioni:

Approccio parametrico: si fanno ipotesi sulla forma delle distribuzioni (es. **distribuzione multinormale**) e si apprendono i parametri fondamentali (**vettore medio, matrice di covarianza**) dal training set.

Approccio non parametrico: si apprendono le distribuzioni dal training set (es. attraverso il metodo **Parzen Window**).

6.4.1 Approccio Parametrico

Generalmente l'approccio parametrico si utilizza quando, oltre ad avere una ragionevole certezza (o speranza) che la forma della distribuzione sia adeguata, la dimensione del training set non è sufficiente per una buona stima della densità. L'approccio parametrico è infatti generalmente caratterizzato da un minor numero di gradi di libertà e il rischio di overfitting dei dati, quando il training set è piccolo, è minore.

6.4.2 Distribuzione Normale Multivariata (Multinormale)

- x_i pattern i-esimo (vettore)
- x_i^j componente j-esima del pattern i-esimo (scalare)

- La densità di probabilità nella distribuzione multinormale ($d > 1$):

$$p(\mathbf{x}) = \frac{1}{(2\pi)^{d/2} |\Sigma|^{1/2}} e^{-\frac{1}{2} (\mathbf{x}-\boldsymbol{\mu})^t \Sigma^{-1} (\mathbf{x}-\boldsymbol{\mu})}$$

dove $\boldsymbol{\mu} = [\mu^1, \mu^2 \dots \mu^d]$ è il **vettore medio** e $\Sigma = [\sigma^{ij}]$ la **matrice di covarianza** ($d \times d$).

Figure 51

μ è il valor medio (media semplice). σ è la deviazione standard, σ^2 è la varianza.

La **Matrice di Covarianza** si indica di solito con Σ ed è la generalizzazione del concetto di varianza al caso di dimensione maggiore di uno. E' una matrice che rappresenta la variazione di ogni variabile rispetto ad altre. E' sempre **simmetrica** (trasposta uguale) e **definita positiva**. La **Matrice Inversa** di una matrice A è pari alla sua Matrice Aggiunta (Trasposta Coniugata) diviso il $\det(A)$.

$|\Sigma|$ e Σ^{-1} sono rispettivamente il determinante e l'inversa di Σ .

La matrice di covarianza è sempre simmetrica e definita positiva, pertanto ammette inversa. Essendo simmetrica il numero di **parametri** che la definisce è $d \cdot (d + 1)/2$

Gli elementi diagonali σ^{ii} sono le varianze dei rispettivi x^i (ovvero $(\sigma^i)^2$); gli elementi non diagonali σ^{ij} sono le covarianze tra x^i e x^j :

- se x^i e x^j sono statisticamente indipendenti $\sigma^{ij} = 0$
- se x^i e x^j sono correlati positivamente $\sigma^{ij} > 0$
- se x^i e x^j sono correlati negativamente $\sigma^{ij} < 0$

Figure 52

6.4.3 Distanza Mahalanobis

La distanza di **Mahalanobis** r tra $\mathbf{x}$ e μ definita dall'equazione:

$$r^2 = (\mathbf{x} - \mu)^t \Sigma^{-1} (\mathbf{x} - \mu)$$

Figure 53

Definisce i bordi a densità costante in una distribuzione multinormale. Tale distanza viene spesso utilizzata in **sostituzione della distanza euclidea**, essendo in grado di **pesare** le diverse componenti tenendo conto dei relativi **spazi di variazione** e della loro **correlazione**

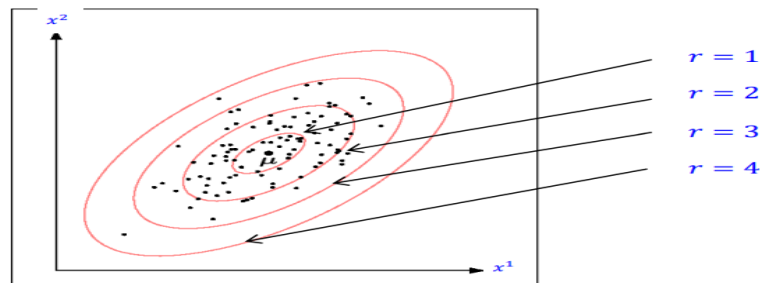
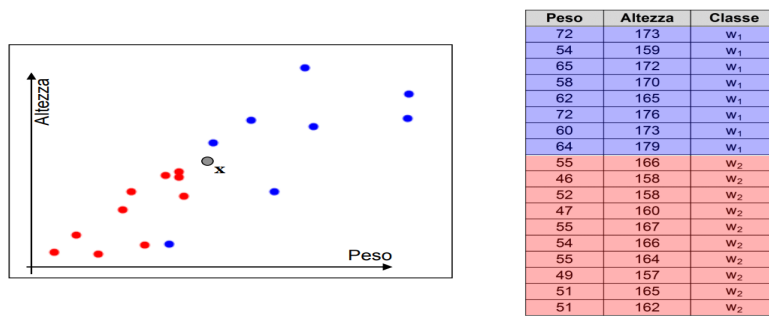


Figure 54

6.4.4 Esempio con Bayes Parametrico (Multinormali)



Obiettivo: stimare la classe di appartenenza del pattern $\mathbf{x}$ (57,168)

Figure 55

Stima dei parametri dal training set:

$$\boldsymbol{\mu}_1 = [63.4, 170.9]^T \quad \boldsymbol{\mu}_2 = [51.5, 162.3]^T \quad \Sigma_1 = \begin{bmatrix} 35.2 & 23.3 \\ 23.3 & 34.9 \end{bmatrix} \quad \Sigma_2 = \begin{bmatrix} 10.1 & 8.9 \\ 8.9 & 13.0 \end{bmatrix}$$

$$p(\mathbf{x} | w_1) = \frac{1}{(2\pi)^{d/2} \cdot |\Sigma_1|^{1/2}} \cdot \exp \left[-\frac{1}{2} (\mathbf{x} - \boldsymbol{\mu}_1)^T \Sigma_1^{-1} (\mathbf{x} - \boldsymbol{\mu}_1) \right]$$

$$p(\mathbf{x} | w_2) = \frac{1}{(2\pi)^{d/2} \cdot |\Sigma_2|^{1/2}} \cdot \exp \left[-\frac{1}{2} (\mathbf{x} - \boldsymbol{\mu}_2)^T \Sigma_2^{-1} (\mathbf{x} - \boldsymbol{\mu}_2) \right]$$

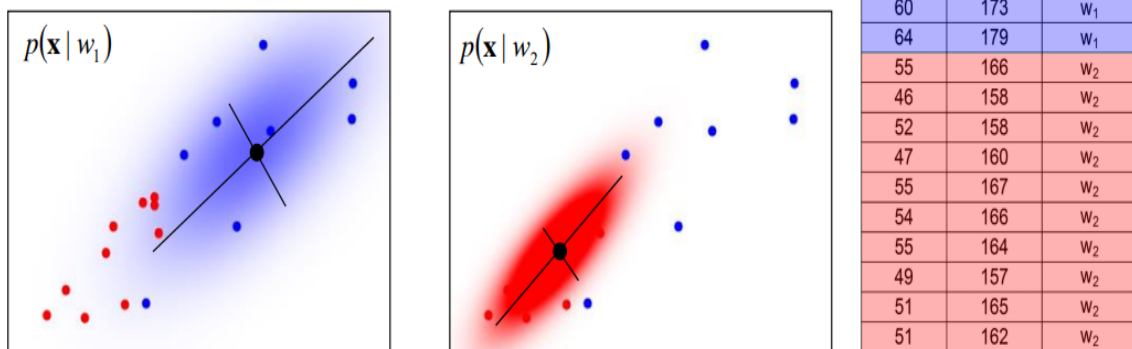


Figure 56

Supponendo di non avere altre informazioni, si possono stimare le probabilità a priori come: $P(w_1) = 8/18$, $P(w_2) = 10/18$

$$p(\mathbf{x} | w_1) = \frac{1}{(2\pi)^{d/2} \cdot |\Sigma_1|^{1/2}} \cdot \exp\left[-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_1)' \Sigma_1^{-1}(\mathbf{x} - \boldsymbol{\mu}_1)\right] = 0.0033$$

$$p(\mathbf{x} | w_2) = \frac{1}{(2\pi)^{d/2} \cdot |\Sigma_2|^{1/2}} \cdot \exp\left[-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_2)' \Sigma_2^{-1}(\mathbf{x} - \boldsymbol{\mu}_2)\right] = 0.0045$$

$$p(\mathbf{x}) = \sum_{i=1}^s p(\mathbf{x} | w_i) \cdot P(w_i) = 0.0040$$

$$P(w_1 | \mathbf{x}) = \frac{p(\mathbf{x} | w_1) \cdot P(w_1)}{p(\mathbf{x})} \cong 0.36$$

$$P(w_2 | \mathbf{x}) = \frac{p(\mathbf{x} | w_2) \cdot P(w_2)}{p(\mathbf{x})} \cong 0.64$$

Peso	Altezza	Classe
72	173	w ₁
54	159	w ₁
65	172	w ₁
58	170	w ₁
62	165	w ₁
72	176	w ₁
60	173	w ₁
64	179	w ₁
55	166	w ₂
46	158	w ₂
52	158	w ₂
47	160	w ₂
55	167	w ₂
54	166	w ₂
55	164	w ₂
49	157	w ₂
51	165	w ₂
51	162	w ₂

Figure 57

Un grande **vantaggio** del classificatore di Bayes, rispetto ad altri classificatori, è legato al fatto che esso produce un valore di output probabilistico, che può essere utilizzato come confidenza.

Infatti, un classificatore può assegnare un pattern $\mathbf{x}$ a una classe w_i con diversi livelli di certezza (o confidenza) che possono essere impiegati per: scartare un pattern in applicazioni open-set **con soglia** costruire un **multi-classificatore**. Se non serve la confidenza:

$$b = \underset{i=1..s}{argmax} \{ p(\mathbf{x} | w_i) \cdot P(w_i) \}$$

Figure 58

Una volta provata una normalità delle distribuzioni, **si stimano** a partire dei dati, vettore medio μ e matrice di covarianza Σ .

Per quanto riguarda le **probabilità a priori** queste possono essere estratte dalle percentuali di campioni che nel training se appartengono alle diverse classi, o in caso di assenza di informazioni possono essere poste tutte uguali tra loro.

Ogni **nuovo pattern da classificare**, è assegnato a una delle possibili classi in accordo con la regola di Bayes nella quale media e covarianza sono ora note.